



INT305 MachineLearning
Lecture11
RecurrentNeuralNetwork

SiyueYu
DepartmentIntelligenceScience
siyue.yu02@xjtlu.edu.cn

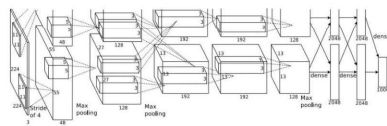
Announcements

- Finish MQ
 - Please kindly be reminded that you are encouraged to provide your feedback to the module INT305 through the Module Feedback Questionnaire.
 - Please take your time to help improving the quality of this module by sharing your comments.

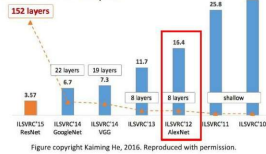
Re-cap

Last Time: CNN Architectures

AlexNet



Revolution of Depth

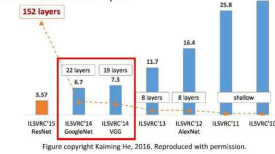


Re-cap

Last Time: CNN Architectures

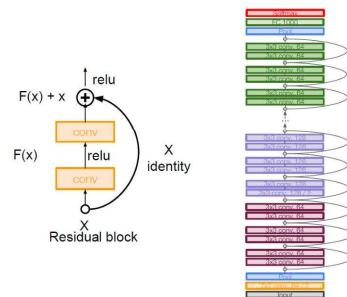


Revolution of Depth



Re-cap

Last Time: CNN Architectures



Revolution of Depth

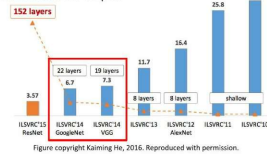
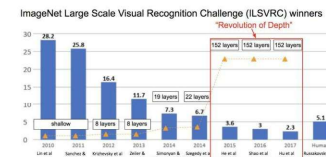


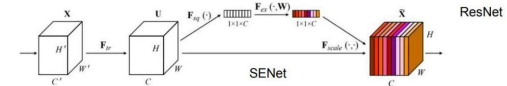
Figure copyright Kaiming He, 2016. Reproduced with permission.

Re-cap

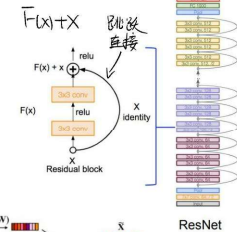
Last Time: CNN Architectures



传统的深度网络就像传话游戏，传了100个人后，最初的信息可能全丢了。ResNet的做法是：每个人在传话处理（ $F(x)$ ）的同时，还给下一个人递一张写着原话的纸条（ x ）。这样即便处理过程中出错了，原始信息也能保留下来。这就让训练152层的“摩天大楼”成为了可能。



ResNet = (3x3卷积 + 3x3卷积) + 跳跃连接



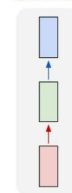
RNN

Today: Recurrent Neural Networks

RNN

“Vanilla” Neural Network

one to one

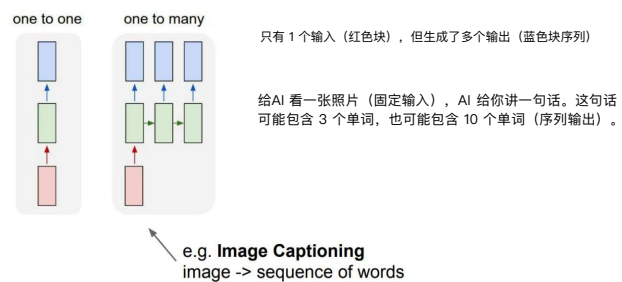


这种网络非常死板。它要求输入是固定大小（比如一张 224x224 的图片），输出也是固定大小（比如一个分类标签）。它处理不了“长短不一”的数据，也记不住上下文

Vanilla Neural Networks

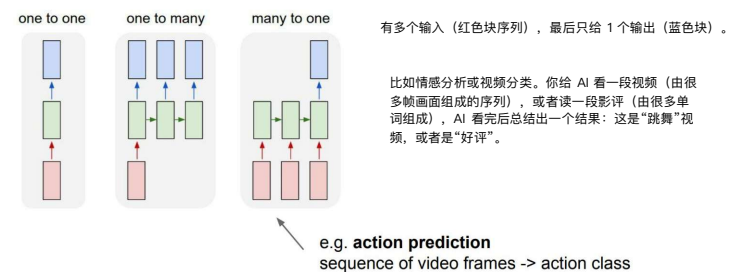
RNN

Recurrent Neural Networks: Process Sequence



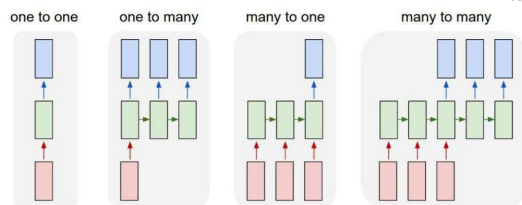
RNN

Recurrent Neural Networks: Process Sequence



RNN

Recurrent Neural Networks: Process Sequence



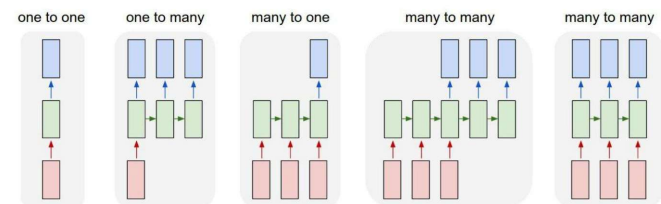
读入一串输入，然后输出一串结果。注意图中是先读完所有绿色块（处理输入），再生成蓝色块（输出）。

类似于机器翻译（Machine Translation）。你输入一句中文（序列），AI 等你说完后，把它翻译成一句英文（序列）。输入和输出的长度可以不一样。

E.g. **Video Captioning**
Sequence of video frames -> caption

RNN

Recurrent Neural Networks: Process Sequence



每一个输入（红色块）立刻对应一个输出（蓝色块）

实时字幕或实时标注。视频一边播，AI 一边给每一帧画面打标签。第一秒是“没人在”，第二秒是“人进来了”，第三秒是“人在跑”。

e.g. **Video classification on frame level**

RNN

以前的神经网络（如 CNN）像是一个只会做填空题的学生（给一个空，填一个词）；而 RNN 像是一个能写作文、能翻译、能实时对话的学生。RNN 的本质就是打破了固定长度的限制，让神经网络学会了处理“过程”和“时间”。

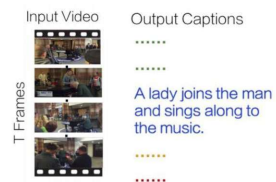
Why existing convnets are insufficient?

Variable sequence length inputs and outputs!

Example task: video captioning

Input video can have variable number of frames

Output captions can be variable length.

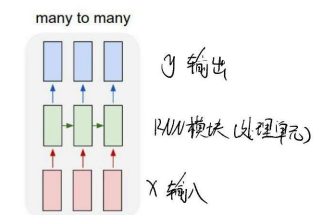


Krishna, Hata, Ren, Fei-Fei, Niebles. Dense captioning Events in Videos. ICCV 2019

RNN

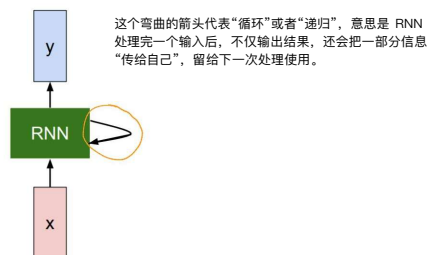
RNN 的基本形态：循环与展开

Let's start with a task that takes a variable input and produces an output at every step



RNN

Recurrent Neural Network

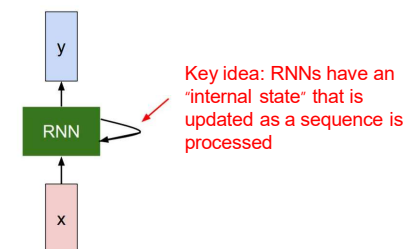


RNN

Recurrent Neural Network

内部状态 (Internal State): 这是 RNN 的灵魂。幻灯片明确指出，RNN 拥有一个“内部状态” (Internal State)，这个状态会随着序列的读取不断更新。

通俗解释: 这就像你的“短期记忆”。当你读到句子的第三个词时，你的脑海里 (内部状态) 已经包含了前两个词的信息。



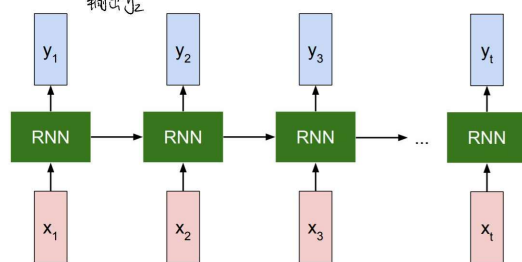
RNN

时刻1: 输入 $x_1 \rightarrow$ RNN $\rightarrow$ 输出 y_1

时刻2: 输入 x_2 + 来自时刻1的记忆 $\rightarrow$ RNN $\rightarrow$ 输出 y_2

...

Unrolled RNN 多个RNN模块按时间展开



RNN

RNN hidden state update

刷新记忆

We can process a sequence of vectors x by applying a **recurrence formula** at every time step:

$$h_t = f_W(h_{t-1}, x_t)$$

new state some function with parameters W old state input vector at some time step

h_t (NewState): 新状态 (今天的记忆)。这是我们要计算的目标。

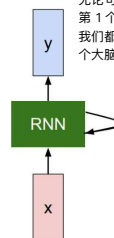
h_{t-1} (OldState): 旧状态 (昨天的记忆)。这是上一步传过来的信息。

x_t (InputVector): 当前输入 (今天的知识)。这是当前时刻进入网络的断数据。

f_W (Function): 处理函数。它带有参数 W , 就像大脑的处理规则。

每一次更新, RNN 都会把“过去的记忆” (h_{t-1}) 和“当前的新闻” (x_t) 融合在一起, 通过一个固定的函数 f_W 进行“搅拌”, 生成“新的记忆” (h_t)。

注意: 这里的 W (参数) 在每一步都是完全相同的。这意味着无论句子多长, 不管是处理第 1 个词还是第 100 个词, 我们都用同一套规则 (同一个大脑) 来处理。



RNN

拥有记忆之后，RNN根据记忆输出反应（输出）的方式。

RNN output generation

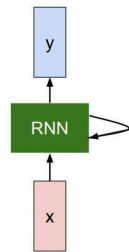
We can process a sequence of vectors x by applying a **recurrence formula** at every time step:

$$\text{output } y_t = f_{w_{hy}}(\text{new state } h_t)$$

output

another function with parameters W .

new state



y_t (Output): 当前输出。比如预测的下一个单词，或者当前帧的分类。

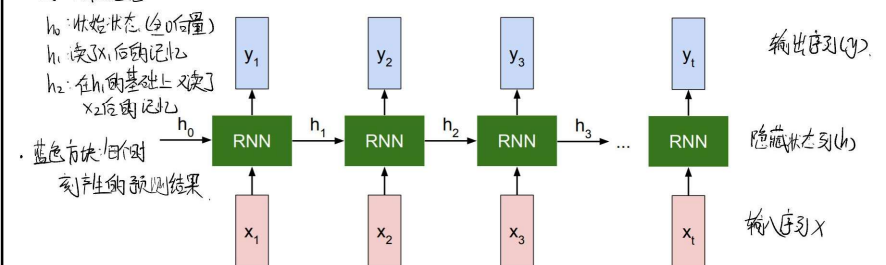
h_t (Current State): 当前状态。也就是刚刚在上一页计算出来的新记忆。

$f_{w_{hy}}$ 输出函数。这是另一套参数（幻灯片中标注为 W_{hy} 或 W_o ），专门负责把“内部记忆”翻译成“外部输出”。

RNN

- 粉色方块 所表示在不同时间点输入的数据（比如句子的第一个词，第二个词...）
- 绿色方块 代表RNN在不同时间点的“长期记忆”

Recurrent Neural Network



RNN

抽象通用公式

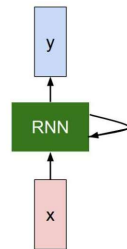
Recurrent Neural Network

We can process a sequence of vectors x by applying a **recurrence formula** at every time step:

$$h_t = f_W(h_{t-1}, x_t)$$

Notice: the same function and the same set of parameters are used at every time step.

不管你处理的是句子的第1个词还是第100个词，你用的大脑结构 (f) 和 脑神经连接 W 是一模一样的。这叫做“权值共享” (Parameter Sharing)，是 RNN 能处理无限长序列的关键



RNN

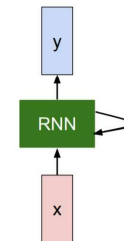
(Simple) Recurrent Neural Network

The state consists of a single "hidden" vector h :

$$h_t = f_W(h_{t-1}, x_t)$$

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

$$y_t = W_{hy}h_t$$

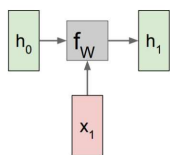


$W_{hh}h_{t-1}$: 回顾过去, 用权重矩阵 W_{hh} (hidden-to-hidden) 对旧状态 h_{t-1} 进行处理
 $W_{xh}x_t$: 接收现在, 用权重矩阵 W_{xh} (input-to-hidden) 对当前输入 x_t 进行处理。
 $+$: 融合
 $\tanh$: 非线性激活: 使用双曲正切函数 ($\tanh$) 将结果压缩到 $0 \sim 1$ 之间, 防止数值在递推循环中爆炸或消散。

Sometimes called a "Vanilla RNN" or an "Elman RNN" after Prof. Jeffrey Elman

RNN

RNN: Computational Graph 计算图



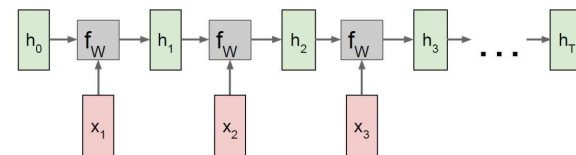
输入 x_1 和初始状态 h_0 进入函数 f_W , 生成了第一个记忆状态 h_1 .

RNN

从单步到循环。

RNN: Computational Graph 参数共享机制

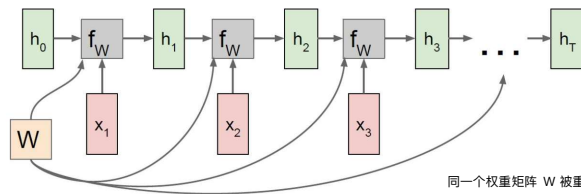
将上面单个的步骤沿着时间轴复制。 h_1 继续与 x_2 结合生成 h_2 , 以此类推到 h_T .



RNN

RNN: Computational Graph

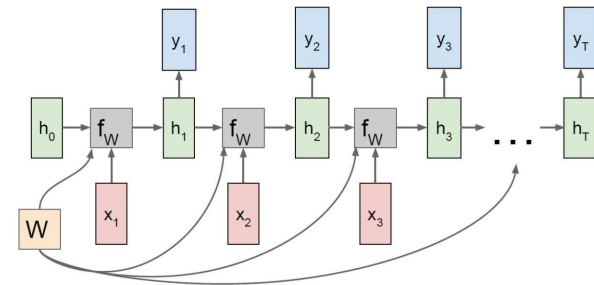
Re-use the same weight matrix at every time-step



同一个权重矩阵 W 被重复使用在每一个时间步上。就像你用同一把锤子去敲所有的钉子，而不是每敲一个钉子换一把锤子。

RNN

RNN: Computational Graph: Many to Many 输入与输出同步模式

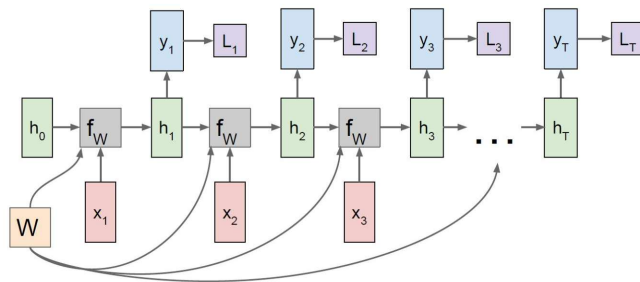


通常用于需要
对每个输入立即做
出反应的任务
(视频逐帧分析)。

RNN

既然每一步都有输出，那么每一步都有可能出错，所以我们为每一个损失(loss)，记为 $L_1, L_2, \dots$ 。

RNN: Computational Graph: Many to Many

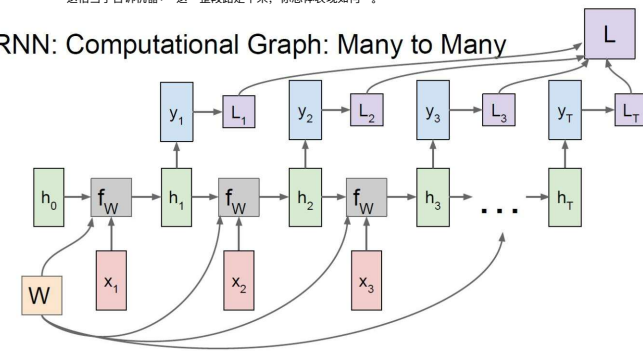


RNN

总损失(Final Loss): 最终的总误差 L 是所有单步误差的总和 (或平均)。

这相当于告诉机器：“这一整段路走下来，你总体表现如何”。

RNN: Computational Graph: Many to Many



RNN

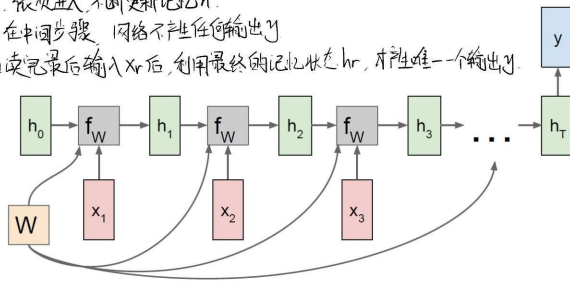
这种模式通常用于总结或分类整个序列的任务（如情感分析：读完整个影评，判断是褒义还是贬义）。

RNN: Computational Graph: Many to One

流程:

- 输入序列: $x_1, x_2, \dots$ 依次进入, 不断更新记忆 h .
- 过程保持沉默: 在中间步骤, 网络不产生任何输出 y .
- 最终爆发: 只有当读完最后输入 x_r 后, 利用最终的记忆状态 h_r , 才产生唯一输出 y .

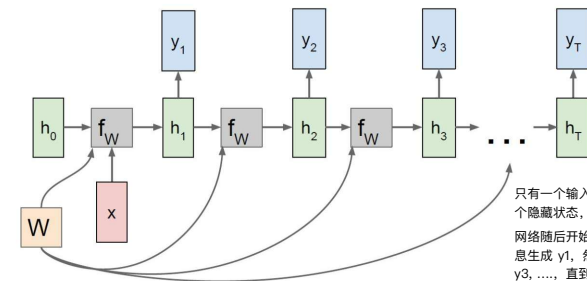
就像侦探听完所有证词后, 最后指认凶手



RNN

这种模式通常用于生成任务（如看图说话：给一张图，生成一句话）。

RNN: Computational Graph: One to Many



只有一个输入 x （通常用来初始化第一个隐藏状态，或者在第一步输入）。网络随后开始“自言自语”：根据初始信息生成 y_1 ，然后基于记忆继续生成 $y_2, y_3, \dots$ ，直到序列结束。

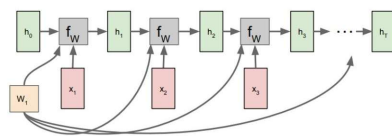
比喻：就像老师给了你一个作文题目（一个输入），你写出了一篇几百字的文章（序列输出）。

RNN

这是最强大的模式，结合了对多和一和一对多。它是机器翻译（如将中文翻译成英文）的核心架构。

Sequence to Sequence: Many-to-one + one-to-many

Many to one: Encode input sequence in a single vector



Sutskever et al., "Sequence to Sequence Learning with Neural Networks", NIPS 2014

第一阶段：编码 (Encoder) – 多对一：

网络先读取整个输入序列（如“你好吗”），但不输出。

目标是把这整句话的信息压缩成一个向量（通常是最终的隐藏状态 h_T ）。这一页明确写道：“Encode input sequence in a single vector”

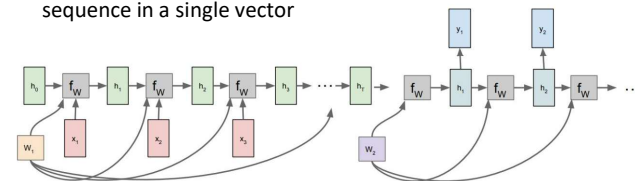
RNN

这种 Encoder-Decoder（编码器-解码器）结构是深度学习在自然语言处理领域的一个里程碑设计

Sequence to Sequence: Many-to-one + one-to-many

Manytoone: Encode input sequence in a single vector

Onetomany: Produce output sequence from single input vector



Sutskever et al., "Sequence to Sequence Learning with Neural Networks", NIPS 2014

第二阶段：解码 (Decoder) – 一对多 (Slide 32):

网络拿着刚才那个压缩好的“上下文向量”，开始生成新的输出序列（如 “How are you”）。

这一页描述为：“Produce output sequence from single input vector”。

RNN

准备阶段

Example: 字符级语言模型。训练 AI 玩“猜下一个字母”的游戏

Character-level LanguageModel

Vocabulary:
[h,e,l,o]

Example training
sequence:
“hello”

任务目标: 我们要教 RNN 学会单词 “hello”。

词汇表(Vocabulary): 这个 AI 的世界里只有 4 个字母: [h, e, l, o]。

编码(One-hot Encoding): 计算机不认识字母, 只认识数字。所以我们用一种叫“独热编码”的方法:

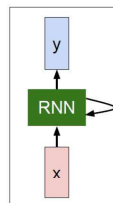
h 变成 [1, 0, 0, 0]

e 变成 [0, 1, 0, 0]

l 变成 [0, 0, 1, 0]

o 变成 [0, 0, 0, 1]

这就像 4 个开关, 哪个字母出现, 就把对应的开关按下去 (变成 1)。



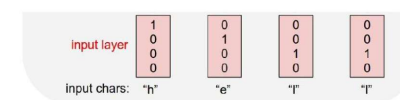
RNN

Example:

Character-level LanguageModel

Vocabulary:
[h,e,l,o]

Example training
sequence:
“hello”



RNN

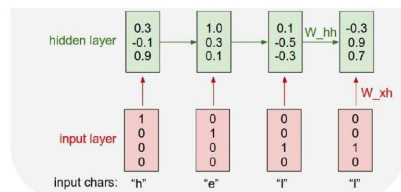
训练阶段

Example: Character-level LanguageModel

Vocabulary:
[h,e,l,o]

Example training
sequence:
"hello"

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$



这里展示了当我们把 "hello" 这个序列输入给 RNN 时，它的内部发生了什么。

右图中展示的信息是：大脑在转动 (Hidden Layer)

我们依次输入 "h", "e", "l", "o".

中间绿色的框是隐藏层。你可以看到里面的数字在不断变化 (比如从 0.3 变成 -0.3 等)

这代表着 RNN 正在根据公式
 $h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$
 不断更新它的记忆。

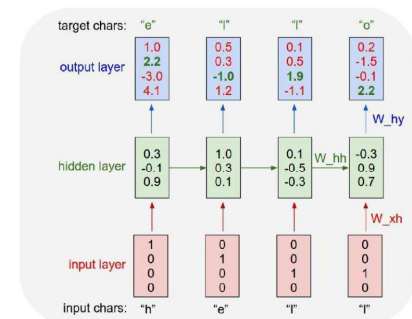
RNN

做出预测

Example: Character-level LanguageModel

Vocabulary:
[h,e,l,o]

Example training
sequence:
"hello"



蓝色的框是输出层，它给出了对下一个字母的打分 (分数越高越好)。

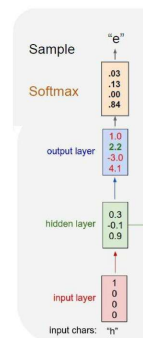
目标 (Targets): 注意顶部的目标字符。当输入 "h" 时，我们希望它预测 "e"; 输入 "e" 时，希望它预测 "l"

RNN

Example: ^{计算概率} Character-level LanguageModel Sampling

Vocabulary:
[h,e,l,o]

Example training
sequence:
"hello"



分数很难直观理解，所以要把它们转换成概率（百分比）。

这就尴尬了：看第一列（输入 "h"），模型预测 "e" 的概率只有 0.13 (13%)，而预测 "o" 的概率高达 0.84 (84%)。

初学者注意：这说明这个模型还没有训练好（或者只是刚开始训练）。此时模型很笨，猜错了。训练的目的就是调整参数，让绿色的 0.13 变大，红色的 0.84 变小。

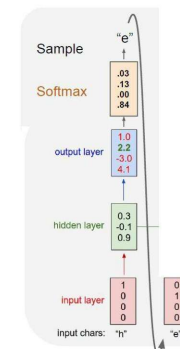
RNN

假设模型已经训练得差不多了，现在我们要让它自己“写字”。这几页展示了一个非常关键的“自循环”过程

Example: Character-level LanguageModel Sampling

Vocabulary:
[h,e,l,o]

At test-time sample
characters one at a time,
feed back to model



我们给它一个 "h"。

模型算出概率分布，然后进行采样 (Sample)。假设它这次运气好，根据概率选中了 "e"。

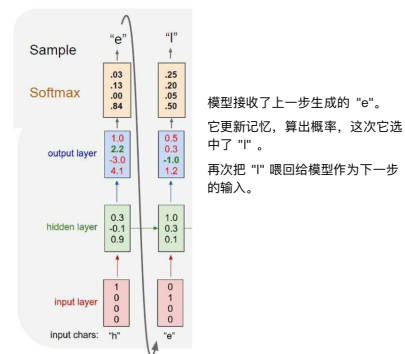
关键动作：这一页最重要的一条线是右边那个弯曲的大箭头。它把刚刚生成的 "e" 抓回来，作为下一个时间步的输入。

RNN

Example: Character-level LanguageModel Sampling

Vocabulary:
[h,e,l,o]

At test-time sample
characters one at a time,
feed back to model

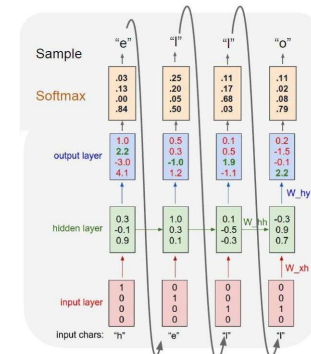


RNN

Example: Character-level LanguageModel Sampling

Vocabulary:
[h,e,l,o]

At test-time sample
characters one at a time,
feed back to model



输入 "h" 猜出 "l"。

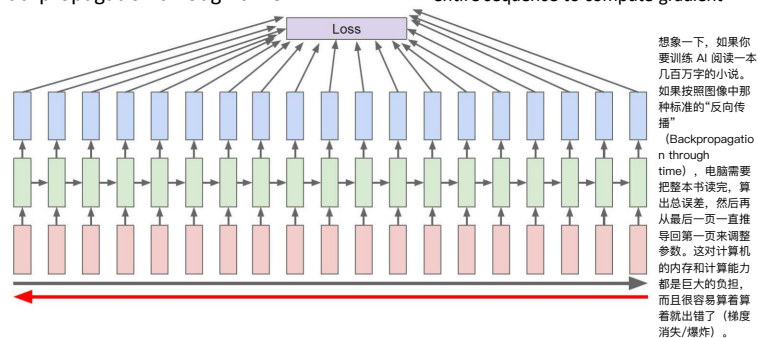
输入 "l" 猜出 "o"。

最终，模型自己输出了完整的序列: "hello"。

RNN

现在有一个非常现实的问题是：如果序列太长了怎么办？

Backpropagation through time

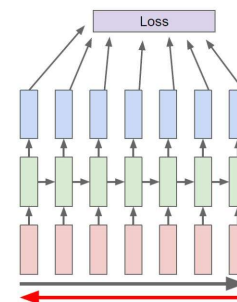


Forward through entire sequence to compute loss, then backward through entire sequence to compute gradient

RNN

可以把这个过程截断学习的过程理解成“分段复习”

Truncated Backpropagation through time



不要试图一口气处理整个序列，而是把它切成一个个小的“块”（Chunks）

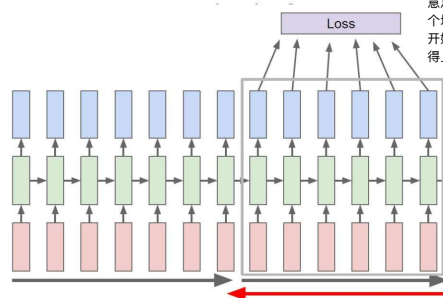
Run forward and backward through chunks of the sequence instead of whole sequence

在序列的小片段上运行前向和后向传播，而不是整个序列

RNN

不仅要往前走，还要懂得回头

Truncated Backpropagation through time



前向传播 (Forward – 黑箭头):

注意文字描述: "Carry hidden states forward in time forever"

意思: 记忆是连续的。虽然我们把数据切开了, 但上一个块结束时的记忆 (h) 会被保留下来, 作为下一个块的开始。就像你看连续剧, 虽然一集一集看, 但你依然记得上一集的剧情。

Carry hidden states forward in time forever, but only backpropagate for some smaller number of steps

反向传播 (Backward – 红箭头):

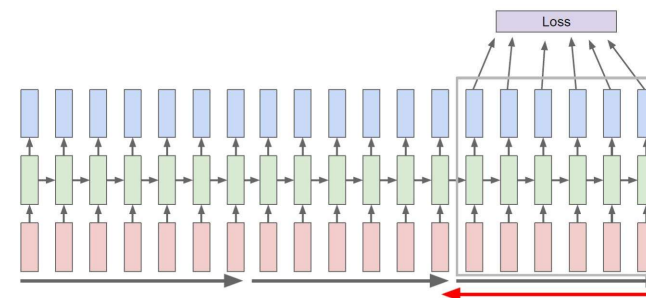
注意文字描述: "but only backpropagate for some smaller number of steps".

意思: 纠错是截断的。当我们计算误差并更新参数时, 我们只在这个灰色方框的范围内往回推。一旦出了这个方框, 就不再继续往回算了。

比喻: 就像你背书, 背完这一段, 发现背错了, 你只纠正这一段的记忆, 而不用跑回去把第一章再背一遍。

RNN

Truncated Backpropagation through time



上图展示了处理完一个块之后的操作。

动作: 灰色方框 (窗口) 向右移动了。

流程: 我们带着上一个块留下的记忆, 进入下一个块, 继续计算损失, 然后在新的块里进行小范围的反向传播。

RNN

RNN tradeoffs

优点：它可以处理任意长度的输入（句子可长可短），并且能利用历史信息。模型的大小不会随着输入的增加而增大。在每个阶段都是用相同的权重，所以对于输入的处理是对称的。

RNN Advantages:

缺点：它是串行计算的，速度慢，而且容易“健忘”（难以获取很久以前的信息）。

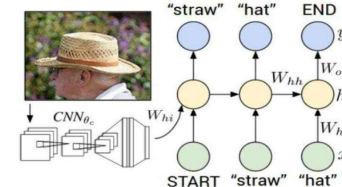
- Can process any length input
- Computation for step t can (in theory) use information from many steps back
- Model size doesn't increase for longer input
- Same weights applied on every timestep, so there is symmetry in how inputs are processed.

RNN Disadvantages:

- Recurrent computation is slow
- In practice, difficult to access information from many steps back

RNN

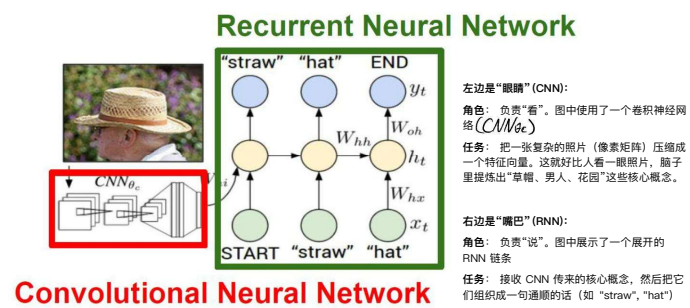
Image Captioning 看图说话



这实际上是一个 Encoder-Decoder (编码器-解码器) 结构

Explain Images with Multimodal Recurrent Neural Networks, Mao et al.
 Deep Visual-Semantic Alignments for Generating Image Descriptions, Karpathy and Fei-Fei
 Show and Tell: A Neural Image Caption Generator, Vinyals et al.
 Long-term Recurrent Convolutional Networks for Visual Recognition and Description, Donahue et al.
 Learning a Recurrent Visual Representation for Image Caption Generation, Chen and Zitnick

RNN



RNN

接下来展示了眼睛是如何工作的。



test image

输入: 一张测试图片(一位戴草帽的老人)。

过程: 图片通过了一系列卷积层(conv-64 到 conv-512)和池化层。这其实就是之前讲过的 VGG-16 网络结构。

结果: 最后通过全连接层(FC-4096), 图片不再是图片了, 变成了一个包含4096个数字的向量。这个向量就是整张图片的“精华浓缩”。

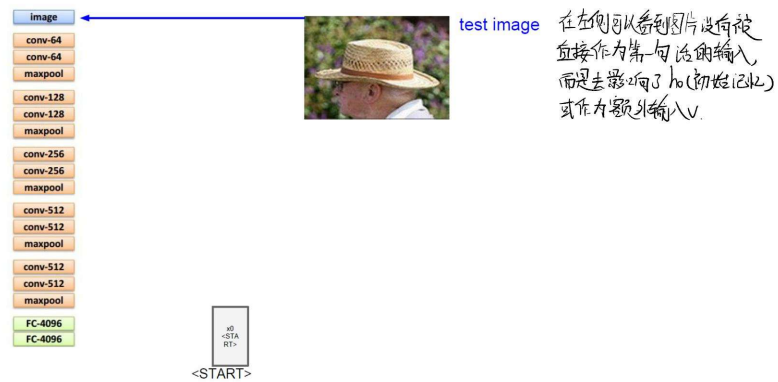
RNN



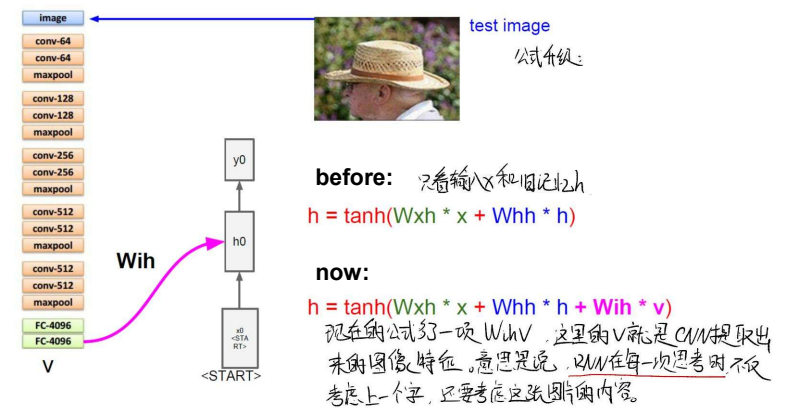
RNN



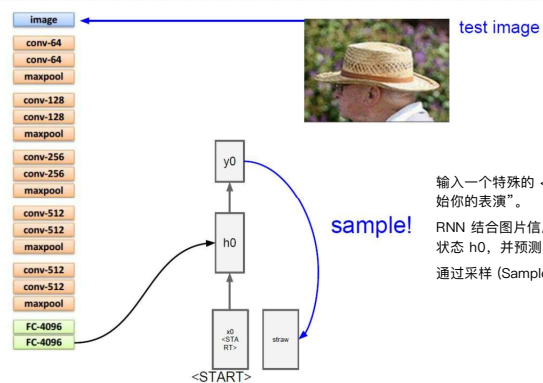
RNN



RNN



RNN

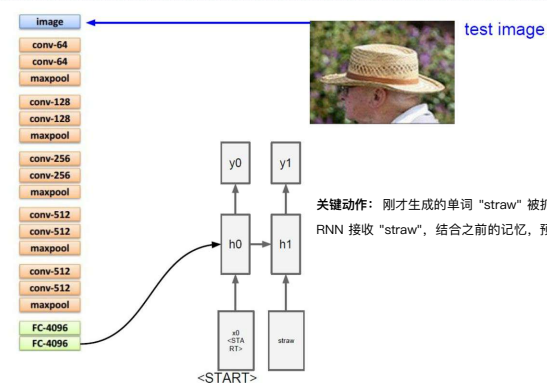


输入一个特殊的 <START> 信号，告诉 RNN “请开始你的表演”。

RNN 结合图片信息 v 和启动信号，计算出第一个状态 h_0 ，并预测出第一个词的概率。

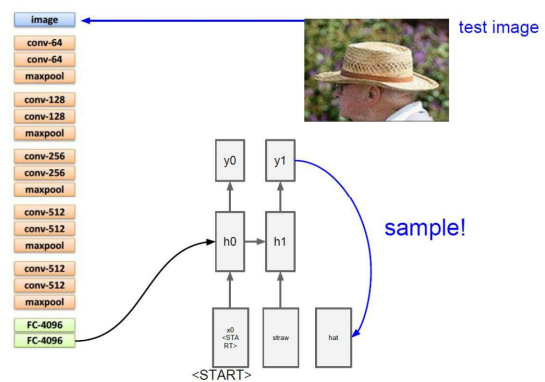
通过采样 (Sample)，它选中了单词 “straw” (草)

RNN

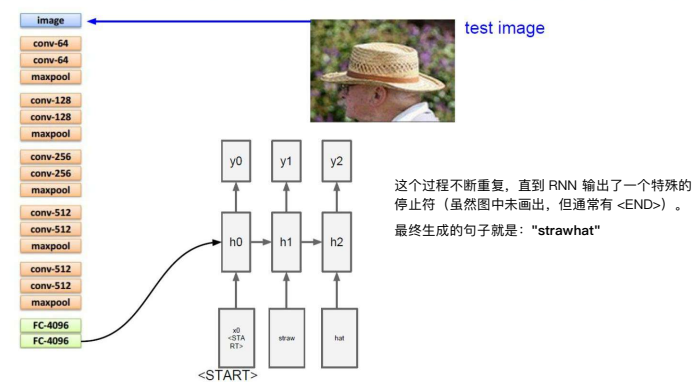


关键动作：刚才生成的单词 “straw” 被抓回来，作为下一个时间步的输入。
RNN 接收 “straw”，结合之前的记忆，预测下一个词是 “hat”(帽子)

RNN



RNN



RNN

Image Captioning: Example Results



A cat sitting on a suitcase on the floor



A cat is sitting on a tree branch



A dog is running in the grass with a frisbee



A white teddy bear sitting in the grass



Two people walking on the beach with surfboards



A tennis player in action on the court



Two giraffes standing in a grassy field



A man riding a dirt bike on a dirt track

Captions generated using [Seq2Seq](#)
All images are [Creative Commons](#)
CC BY-SA 4.0 licensed. This work
is licensed under a [Creative Commons](#)
Attribution-ShareAlike 4.0 International License

RNN

Image Captioning: Failure Cases



A woman is holding a cat in her hand



A person holding a computer mouse on a desk



A woman standing on a beach holding a surfboard



A bird is perched on a tree branch



A man in a baseball uniform throwing a ball

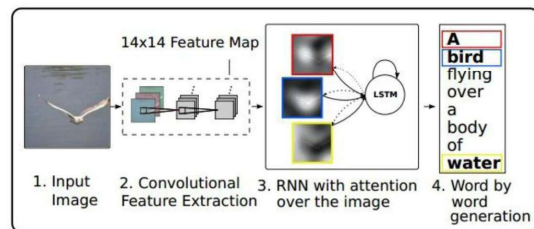
Captions generated using [Seq2Seq](#)
All images are [Creative Commons](#)
CC BY-SA 4.0 licensed. This work
is licensed under a [Creative Commons](#)
Attribution-ShareAlike 4.0 International License

RNN

之前的模型（第 45-56 页）有一个缺点：它试图把整张图片压缩成一个固定的向量传给 RNN。这就好比让你盯着一张复杂的图看一眼，然后闭上眼睛描述它，不仅容易忘，还容易漏掉细节。

Image Captioning with Attention

RNN focuses its attention at a different spatial location when generating each word

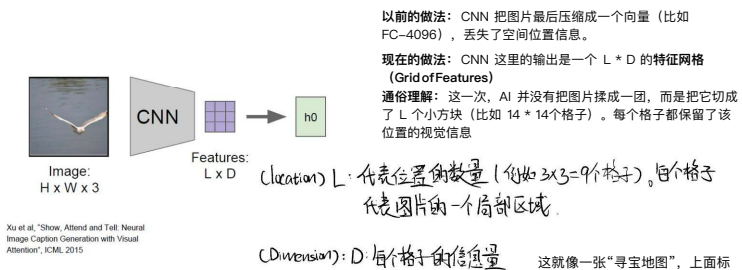


Xu et al, "Show, Attend, and Tell: Neural Image Caption Generation with Visual Attention", ICML, 2015
Figure copyright Kelvin Xu, Jimmy Lei Ba, Jamie Kiros, Kyunghyun Cho, Aaron Courville, Ruslan Sazhukhinov, Richard S. Zemel, and Yoshua Bengio, 2015. Reproduced with permission.

Attention 机制改变了这一点。它让 AI 学会了“边看边说”。每说一个词，它都会把目光聚焦在图片的不同位置。

RNN

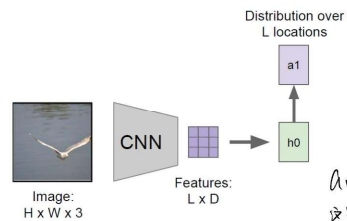
Image Captioning with Attention



Xu et al, "Show, Attend and Tell: Neural Image Caption Generation with Visual Attention", ICML, 2015

RNN

Image Captioning with Attention



Xu et al., "Show, Attend and Tell: Neural Image Caption Generation with Visual Attention", ICML, 2015

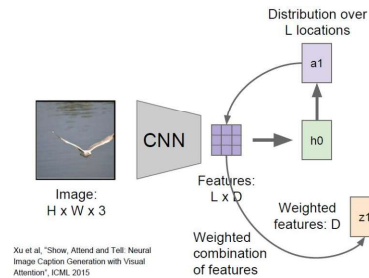
计算注意力 (Distribution over L locations):

RNN 的大脑 (当前状态 h_0) 会去查看这 L 个格子。它会问自己: "我现在刚开始说话, 我该看哪里?" 计算结果是一个分布 a_1 , 也就是给每个格子打分。分数高的地方, 就是当前最重要的地方。

a_1 是 distribution over L locations. 这是一个概率分布 (比如: 左上角 1%, 中间 8%, 右下角 1%)。则代表 8% 的注意力在中间的格子上。

RNN

Image Captioning with Attention



Xu et al., "Show, Attend and Tell: Neural Image Caption Generation with Visual Attention", ICML, 2015

(根据刚才的注意力分布 a_1 , 对 L 个格子进行加权求和)

根据刚才的打分 (a_1), 模型把这 L 个格子的信息加权融合。生成一个加权特征向量 z_1 。

核心公式: $z = \sum p_i v_i$

比喻: 这就像聚光灯打在了分数高的区域, 其他区域变暗了, 只有聚焦区域的信息被抽取出来传给了 RNN。

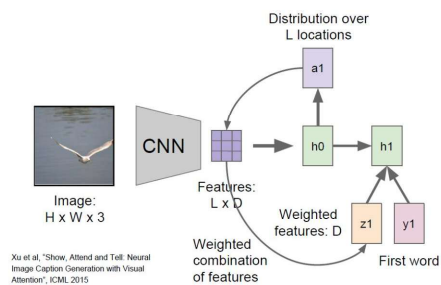
V_i 代表第 i 个位置的视觉特征向量。

CNN 把图片切成了 L 个格子, 每个小格子中的信息都被压缩成一个数字向量。

p_i : 代表模型对第 i 个位置的注意力权重 (0~1 之间的概率值)。如果 p_i 很大, 那么代表模型觉得第 i 个格子对当前要生成的词非常重要。

RNN

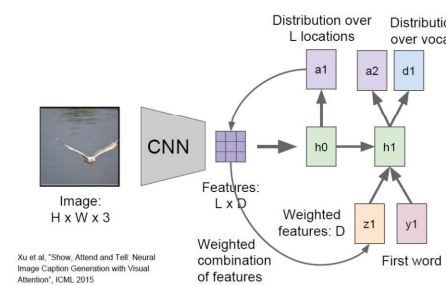
Image Captioning with Attention



RNN接收这个聚集的信息 z_1 ,更新自己的记忆 h_1 ,然后输出第一个词 y_1 (比如A).
这个词被选中 (采样得到 d_1)并准备作为下一步输入。

RNN

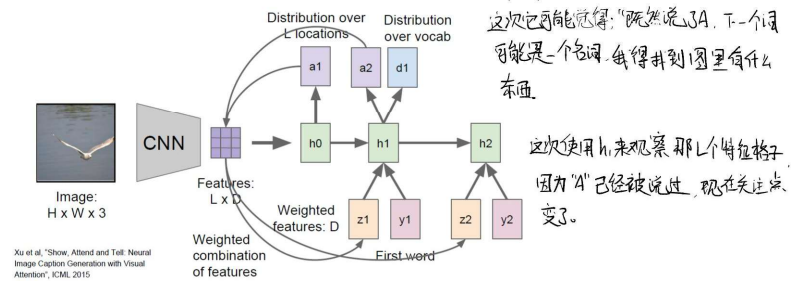
Image Captioning with Attention



h_1 负责计算词汇表的概率分布 d_1 ,并选出第一个单词 y_1 (比如"A").
同时 h_1 也准备对去计算下一步的注意力。

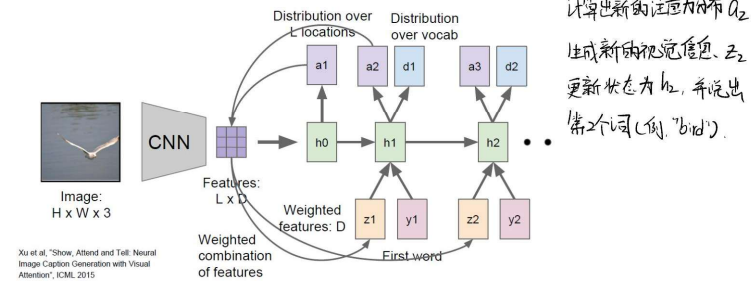
RNN

Image Captioning with Attention



RNN

Image Captioning with Attention



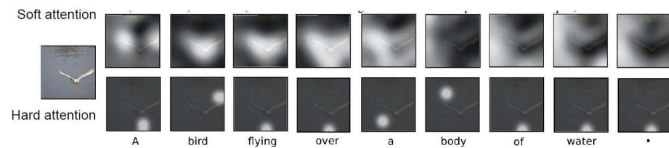
RNN

Image Captioning with Attention

聚光灯效果

当生成单词 "bird" 时，图片中白色的高亮区域（关注点）集中在鸟的身体上。

当生成单词 "water" 时，高亮区域转移到了背景的水面。



Xu et al., "Show, Attend, and Tell: Neural Image Caption Generation with Visual Attention", ICML 2015
 Figure copyright Kelvin Xu, Jimmy Lei Ba, Jianbo Ji, Kyunghyun Cho, Aaron Courville, Razvan Saeedhodini, Richard S. Zemel, and Yoshua Bengio, 2015. Reproduced with permission.

RNN

Image Captioning with Attention

比如生成 "frisbee" (飞盘) 时，AI 就会盯着图片里的飞盘看；生成 "stop sign" (停车标志) 时，就会盯着红色的牌子看。



Xu et al., "Show, Attend, and Tell: Neural Image Caption Generation with Visual Attention", ICML 2015
 Figure copyright Kelvin Xu, Jimmy Lei Ba, Jianbo Ji, Kyunghyun Cho, Aaron Courville, Razvan Saeedhodini, Richard S. Zemel, and Yoshua Bengio, 2015. Reproduced with permission.

RNN

RNN 在训练（反向传播）时的一个致命弱点：**梯度消失 (Vanishing Gradients)**。

Long Short Term Memory (LSTM)

Vanilla RNN

$$h_t = \tanh \left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix} \right)$$

LSTM

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

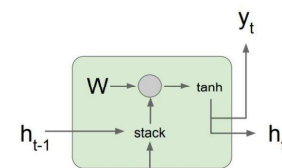
$$h_t = o \odot \tanh(c_t)$$

Hochreiter and Schmidhuber, "Long Short Term Memory", Neural Computation 1997

RNN

Vanilla RNN Gradient Flow

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML, 2013



$$h_t = \tanh(W_{hh}h_{t-1} + W_{hx}x_t)$$

$$= \tanh \left(\begin{pmatrix} W_{hh} & W_{hx} \end{pmatrix} \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix} \right)$$

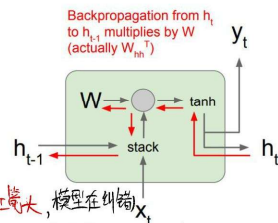
$$= \tanh \left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix} \right)$$

前向传播(黑箭头, 模型在学习) x_t
信息从左边(旧记忆 h_{t-1})和下面(新输入 x_t)进来, 经过中间处理器 (stack $\rightarrow$ $W \rightarrow$ $\tanh$),
生成右边的新记忆 h_t 和上面的输出 y_t

RNN

Vanilla RNN Gradient Flow

Bengio et al. "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al. "On the difficulty of training recurrent neural networks", ICLR, 2013



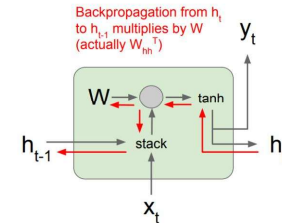
反向传播 (红箭头, 模型在纠错)
假设 y_t 错了, 我们需要算出是谁的责任, 然后让红箭头把责任信号 (也就是梯度) 传递回去。

$$\begin{aligned} h_t &= \tanh(W_{hh}h_{t-1} + W_{xh}x_t) \\ &= \tanh\left((W_{hh} \quad W_{xh}) \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \\ &= \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \end{aligned}$$

RNN

Vanilla RNN Gradient Flow

Bengio et al. "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al. "On the difficulty of training recurrent neural networks", ICLR, 2013



$$\begin{aligned} h_t &= \tanh(W_{hh}h_{t-1} + W_{xh}x_t) \\ &= \tanh\left((W_{hh} \quad W_{xh}) \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \\ &= \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \end{aligned}$$

$$\frac{\partial h_t}{\partial h_{t-1}} = \tanh'(W_{hh}h_{t-1} + W_{xh}x_t)W_{hh}$$

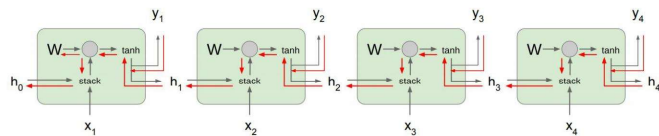
乘法的内容取决于两样东西: W_{hh} : 当前权重矩阵, $\tanh'$: $\tanh$ 函数的导数 (变化率)。

当我们想知道“现在的错误”与“过去的记忆”有什么关系时 (即 $\frac{\partial h_t}{\partial h_{t-1}}$), 我们需要做乘法运算。

RNN

Vanilla RNN Gradient Flow

Bengio et al. "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al. "On the difficulty of training recurrent neural networks", ICML, 2013



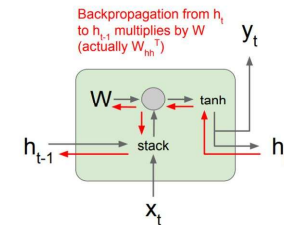
$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

我们在 $t=1, 2, 3, 4$ 每个时刻都有输出, 总损失 (loss) 是所有时刻错误的综合。

RNN

Vanilla RNN Gradient Flow

Bengio et al. "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al. "On the difficulty of training recurrent neural networks", ICML, 2013



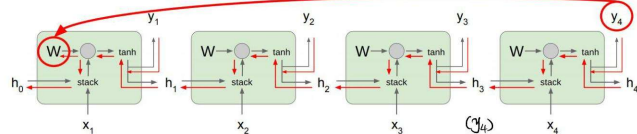
$$\frac{\partial h_t}{\partial h_{t-1}} = \tanh'(W_{hh}h_{t-1} + W_{hx}x_t)W_{hh}$$

$$\begin{aligned} h_t &= \tanh(W_{hh}h_{t-1} + W_{hx}x_t) \\ &= \tanh\left((W_{hh} \quad W_{hx}) \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \\ &= \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \end{aligned}$$

RNN

Vanilla RNN Gradient Flow

Gradients over multiple time steps:



$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \frac{\partial h_T}{\partial h_{t-1}} \dots \frac{\partial h_1}{\partial W}$$

假设我们在第4步犯了一个错误, (比如把对最后的
问号写成了句号)

我们需要告诉第1步的权重 W: “你在处理 x 的时候
有点问题, 导致我现在第4步出错了, 请改正。”

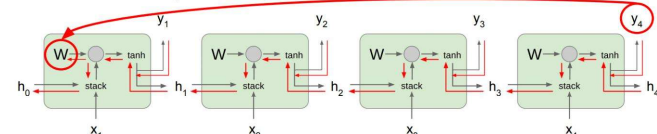
这个误差信号必须逆流而上, 从 h_4 传回 h_3, 再传回 h_2,
最后传回 h_1.

RNN

致命弱点: 信号衰减.

Vanilla RNN Gradient Flow

Gradients over multiple time steps:



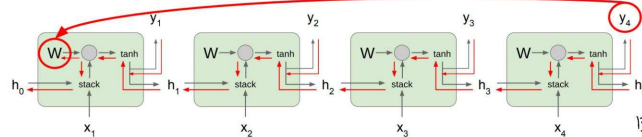
$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \frac{\partial h_T}{\partial h_{t-1}} \dots \frac{\partial h_1}{\partial W} = \frac{\partial L_T}{\partial h_T} \left(\prod_{t=2}^T \frac{\partial h_t}{\partial h_{t-1}} \right) \frac{\partial h_1}{\partial W}$$

RNN

Vanilla RNN Gradient Flow

Gradients over multiple time steps:



$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W} \quad \boxed{\frac{\partial h_t}{\partial h_{t-1}} = \tanh'(W_{hh}h_{t-1} + W_{xh}x_t)W_{hh}}$$

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \frac{\partial h_t}{\partial h_{t-1}} \cdots \frac{\partial h_1}{\partial W} = \frac{\partial L_T}{\partial h_T} \left(\prod_{t=2}^T \boxed{\frac{\partial h_t}{\partial h_{t-1}}} \right) \frac{\partial h_1}{\partial W}$$

回顾刚才的“单步
纠错”，每往回退一步
就要做一次乘法，
(乘以W和 tanh')
如果要在回退T步，根据链
式法则，我们需要把中间
每一步梯度连接起来。

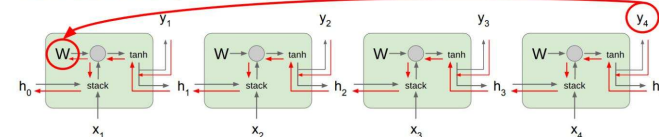
RNN

核心问题：tanh 函数的导数总是小于 1 的（大部分时间甚至非常小），而且权重 W 如果初始化得不好也可能很小。

结果：当你把一堆小于 1 的数字（比如 0.5）连续相乘很多次（0.5 * 0.5 * 0.5 ...），结果会迅速接近于 0。

Vanilla RNN Gradient Flow

Gradients over multiple time steps:



$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W} \quad \text{Almost always } < 1 \quad \text{Vanishing gradients}$$

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \left(\prod_{t=2}^T \boxed{\tanh'(W_{hh}h_{t-1} + W_{xh}x_t)} \right) W_{hh}^{T-1} \frac{\partial h_1}{\partial W}$$

梯度消失 (Vanishing Gradients): 这意味着，从第 4 步发出的强烈的“纠错信号”，传到第 1 步时已经微弱到几乎没有了。

后果：第 1 步的参数 W 根本收不到信号，它觉得自己没错，所以不进行修改。这就导致 RNN 无法学习到长距离的依赖关系（记不住开头发生的事）。

LSTM

Long Short Term Memory (LSTM) 长短期记忆网络

普通RNN有一个

状态 h 。它既负责记忆

又负责输出，压力很大，容易忘记

公式也很简单， $h = \tanh(\dots)$ 。一直在乘法，容易衰减

$$h_t = \tanh \left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix} \right)$$

Hochreiter and Schmidhuber, "Long Short Term Memory", Neural Computation

LSTM

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

LSTM引入一个新的核心变量： C_t (cell state)

h_t (Hidden State): 还是原来的工作记忆，负责当前输出

C_t (Cell State): 这是一个长期记忆，贯穿了整个流程
信息在这里主要进行加法，梯度难消失。

其诞生的目的就是
为了解决“梯度消失”的
问题，为RNN装上了一个
可以长期保存信息的
保险箱。

LSTM

为了保护 C_t 不被随意涂改，LSTM 设计了4个 gates

输入还是这样子：旧的 h_{t-1} 和新的 x_t 堆叠在一起，乘以权重矩阵 W 。但是输出分成了4份。

Long Short Term Memory (LSTM)

[Hochreiter et al., 1997]

① input gate: 决定要写多少东西进去。

Sigmoid(6)。输出0到1，表示“完全不写”

或“全写进去”

② forget gate: 决定以前
的内容要擦掉多少。

Sigmoid(6)。输出0 (忘记)
或 1 (记住)。

③ Output gate: 决定我要
把心里的秘密展示

多少给外界看，

Sigmoid(6)。

④ Gate gate / Candidate

原本打算写入的新信息 (草稿)。

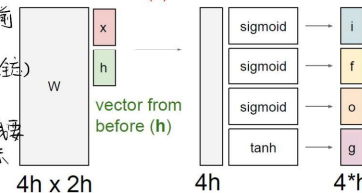
Tanh。注意只有这里的激活函数是 tanh，其他都是 sigmoid(6)。因为记忆有具体的数据信息 (有正有负)，而不是像上面那三个只是0-1开关。

i: Input gate, whether to write to cell

f: Forget gate, Whether to erase cell

o: Output gate, How much to reveal cell

g: Gate gate (?), How much to write to cell



$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

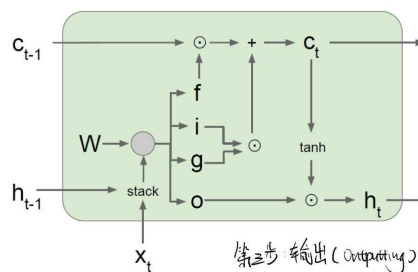
$$h_t = o \odot \tanh(c_t)$$

LSTM

LSTM 内部完整工作流程，最重要的部分是从 C_{t-1} 到 C_t 的线。

Long Short Term Memory (LSTM)

[Hochreiter et al., 1997]



第一步：遗忘 (Forgetting): $C_{t-1} \times f$

拿着上一时刻的细胞本 (C_{t-1})，根据遗忘门 f 的指令，把过期的信息擦掉。

第二步：写入 (Writing): $+ (i \times g)$

把新写的草稿 (g) 根据输入门 (i) 批准，加到笔记本上。注意这里是“+”号。类似 ResNet 设计。

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

因为加法，梯度在反向传播的时候，可以无损通过。

$$C_t = f \odot C_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(C_t)$$

第三步：输出 (Output): $h_t = o \times \tanh(C_t)$

最后，根据当前内容 C_t ，经过输出门的筛选，决定当前时刻对外表现 (h_t)。

它不再盲目地每次都把旧记忆和新输入揉在一起。相反，它像一个严谨的档案管理员：

1. 先看一眼旧档案，删掉没用的 (f)。
2. 再看一眼新文件，添加有用的 (i, g)。
3. 这个档案 (C) 主要通过加法更新，所以哪怕过了 100 年 (100 个时间步)，核心信息依然能保留下来。
4. 最后再决定给领导汇报什么 (o, h)。